Project No: A1068-221

Blockchain-based Relayer Services: Batching and Submitting Multiple Energy Transactions in a Meta-Transaction

Submitted by: Ma Siteng (U1923700D)

Supervisor: Prof. Gooi Hoay Beng
Co-supervisor: Dr. Yang Jiawei
Examiner: Prof. Amer Mohammad Yusuf Mohammad Ghias

School of Electrical & Electronic Engineering

A final year project report presented to the Nanyang Technological University
in partial fulfilment of the requirements of the degree of
Bachelor of Engineering

2022/2023

Table of Contents

Abstract

Peer-to-peer energy trading has seen an increase of popularity with the adoption of smart grid and distributed energy resources in current days. Blockchain, a distributed ledger governed by consensus protocol, is deemed as a solid foundation to build the Peer-to-peer energy trading network on top of it where traceable and verifiable transaction logs become feasible.

However, even if public blockchain is permissionless, meaning everyone can join the blockchain, it is still a closed ecosystem where everything must be settled in the crypto-native way. One typical example is users need to pay 'gas fee' in the blockchain's native token to launch transaction on it. This may not be the problem for the crypto adopters, but it causes significant onboarding issues for users without prior experience. So does it to the users of blockchain-based Peer-to-peer energy trading network.

Moreover, even if the trading system is built based on private or consortium blockchain instead of public blockchain, which means users do not need to pay 'gas fee', the trading efficiency still requires improvement.

To tackle these problems mentioned above, this project presents a full stack relayer service that eliminates prosumers' need to pay gas fee by themselves during the settlement stage in the trading process, and greatly improves the trading efficiency by batching multiple payment requests in one transaction. Prosumers can handle the payment requests to the relayer which launches the payment transactions for the prosumers.

Acknowledgements

I would like to express my deepest gratitude to my supervisor Associate Professor Gooi Hoay Beng, for giving me the opportunity to take up this project so that I could learn more about blockchain's application in the P2P energy trading sector. I am truly grateful for his firm support and sincere advice during this journey.

I would also like to thank my co-supervisor, Dr. Yang Jiawei, for patiently guiding me through the project requirements, ideation, and implementation. His great knowledge in both blockchain and energy trading has supported me a lot in terms of understanding the context and carrying out the design along the way.

Finally, I would like to express my appreciation towards my parents. I cannot pursue a degree and accomplish all these without their unconditionally support and encouragement.

It is my honour to have met everyone in this journey and thank you all once again.

List of Figures

List of Tables

Chapter 1 Introduction

1.1 Background

Nowadays, with more and more distributed energy resources (DERs) and storage devices being adopted, energy consumers evolve to prosumers which can not only consume energy, but also produce energy on one's own. The emergence of prosumer-centric power system spawns the idea of Peer-to-Peer (P2P) energy trading which enables prosumers to sell the excess generated energy directly to other prosumers with energy deficiency, without the participation of intermediaries [1].

Blockchain, a distributed ledger powered by consensus algorithm, is a natural match up with P2P energy trading since it can record energy transactions with verifiability and traceability and offer programmable smart contract that coordinates order matching and payment settlement in the trading process.

1.2 Problem Statement

To send transactions (i.e. interact with smart contract) in P2P energy trading network based on public blockchain, prosumers need to pay a commission fee called 'gas' in blockchain's native token so that blockchain validator can confirm, execute and record the transaction on blockchain. This causes great inconvenience for prosumers without prior experience in crypto. To send energy transactions, they need to purchase the native token from crypto exchanges and figure out how to safely hold it in digital wallets. This hinders the further adoption of blockchain in energy trading.

In addition, the trading efficiency in P2P energy trading network based on blockchain still has room for improvement. For trading network based on private or consortium blockchain, prosumers may not need to pay ‘gas fee’ to launch transactions, but they need to interact with the blockchain directly to submit auction price or payment transactions. Considering the scale of private blockchain is not very large, with only a few nodes running in the network, the private blockchain is prone to congestion if there are a large number of transactions taking place at the same time.

1.3 Objectives and Scope

This project aims to utilize the concept of Meta-Transaction and batching mechanism to build a full stack relayer service that makes the p2p energy trading system based on blockchain much easier to use and improves the trading efficiency. The relayer bundles multiple energy transactions and submits them to blockchain at the same time for prosumers in the settlement stage. Relayer will pay gas fee for prosumers.

The scope of this project is a programming implementation of a web-based relayer service focusing on settlement stage in the energy trading market, where consumers pay sellers a specific amount of prescribed token based on smart contract. The relayer service can be used in scenario where energy tokenization and transfer take place as well. This project does not cover the economic model of prosumers paying fee to relayer. We assume the relayer is offering service free of charge.

Chapter 2 Literature Review

2.1 P2P Energy Trading

P2P energy trading is invented to align with the trend including the popularization of distributed renewable energy resources and the increase in the number of prosumers including dwellings, factories, schools, and offices in local electricity network [2]. The progress in information systems accelerates the birth of P2P energy trading [3].

In traditional power grid systems, large-scale electricity producers are the main entity that generates power, then sells and transmits it to regional power grids consisting of consumers. Both the producing and trading of electricity are a unidirectional behaviour, as shown in Figure 1(a).

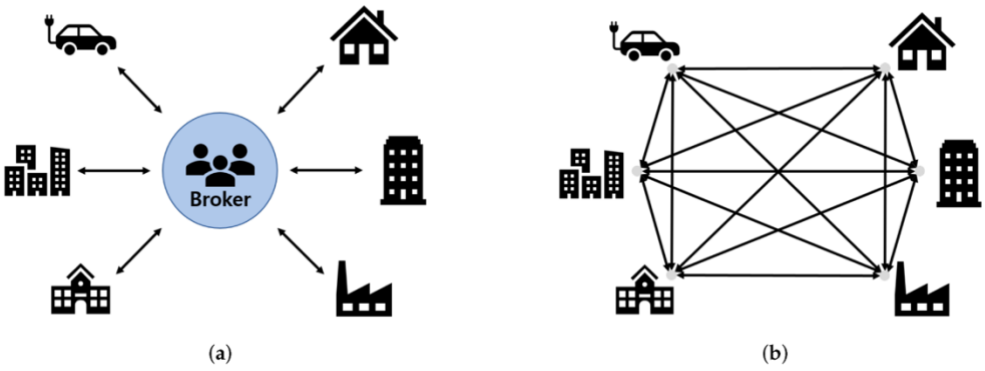

Figure 1 Traditional energy trading vs P2P energy trading

While in P2P energy trading, individual prosumers own distributed energy resources like solar photovoltaic panels which enable them to produce energy as well. The energy generating devices connects with the grid and trade the excess energy produced to grid or other prosumers. This forms a bilateral market, and the transaction of energy becomes multi-directional, as shown in Figure 1(b).

This article [4] summarizes the typical procedure of P2P energy trading on marketplace into three stages, as shown in Figure 2:

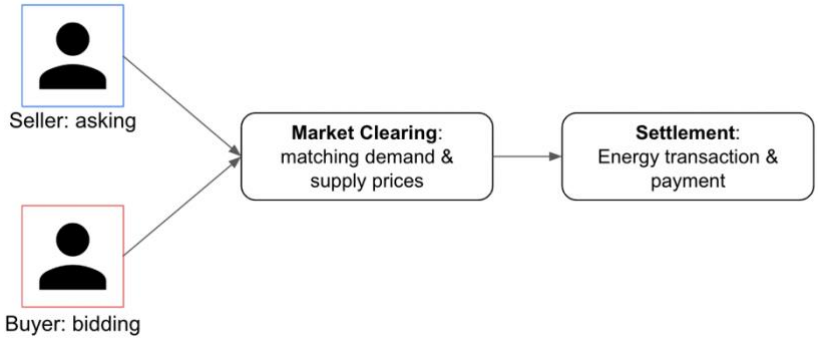

Figure 2 Procedure of P2P energy trading marketplace

2.2 Blockchain

Blockchain works as a distributed ledger that records transactions happening on it. It is governed by consensus mechanism maintained by the validators or miners. A transaction on blockchain will only be valid after it is confirmed by the validator and then can be recorded in a newly created block, hence each block consists of a bunch of transactions. These blocks connect linearly in time order, and eventually they form a chain, as shown in Figure 3. Leveraging cryptography including public-key encryption, hash function, and Merkle tree, the data structures of blockchain, from the transactions packaged by each block to the blocks connected in one chain, are organized in an immutable way, together with other special attributes including transparency, traceability, and verifiability for the data it records [5].

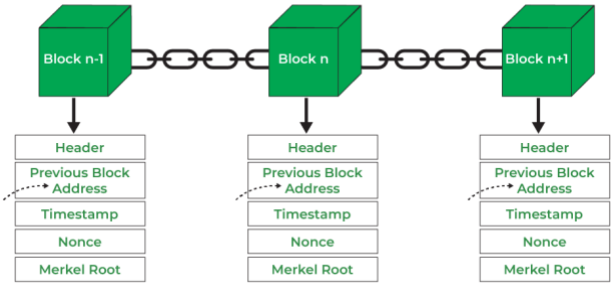

Figure 3 Blockchain architecture breakdown

Invented in 2009, blockchain by then was merely a P2P digital payment network where users can only transfer the native coin Bitcoin and perform mining to earn bitcoin. After going through a series of evolution, much more powerful blockchains have been developed, where developers can build decentralized applications for multiple usages including tokenization, decentralized finance (DeFi), gaming, social etc. Other than these native applications, we are witnessing more and more application of blockchain in other industries such as IoT, supply chain, energy, cyber security [6]. One typical example of the new generation blockchain is Ethereum which offers smart contract programmability.

Other than public blockchains such as Bitcoin and Ethereum, variants such as private blockchain and consortium blockchain also emerge. They sacrifice the scale, permissionlessness, and decentralization of blockchain to some extent in exchange for a higher scalability.

2.2.1 Smart Contract

Smart contract refers to immutable computer programs that run deterministically on blockchain. In the context of Ethereum, smart contract is written in Solidity, an object-oriented programming language. The smart contract runs passively, which means it can only be triggered by externally owned account (EOA) or other contracts submitting transactions. The deployment and execution of smart contract is performed by Ethereum Virtual Machine (EVM).

Once the deployed smart contract on Ethereum is triggered, it requires a gas fee payment in Ether. Gas is a resource restricting the maximum amount of computation resource that Ethereum allows a transaction to consume to avoid Denial of Service (DoS) attack such as non-stop iteration. Users can specify the amount of gas to pay to

reward validators' work as bookkeepers. With higher gas fee payment, the validator is more likely to put this transaction in front of other transactions for confirmation and execution.

2.2.2 Digital Signature

Blockchain adopts public key cryptography for account creation. The public key algorithm Ethereum adopts is Elliptic Curve Digital Signature algorithm (ECDSA) secp256k1. Its visualisation is shown in Figure 4.

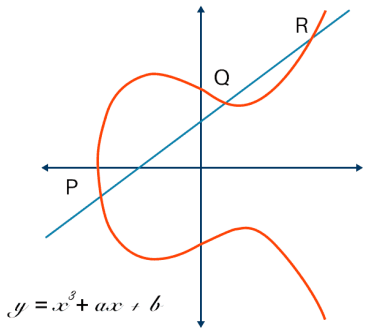

Figure 4 ECDSA curve

When an account is created in Ethereum, a public/private key pair is created using secp256k1. After alternation, the account address is derived from the public key, hence the address of an Ethereum account is the representation of a secp256k1 public key. And since the private key is bind with the public key, the address represents the account's ownership of the respective private key. The private key is used to sign the transaction whenever the user submit to Ethereum. The signature is able to guarantee non-repudiation and prove the transaction data cannot be modified.

Cryptocurrency wallet stores the account private key and hence it can be used to sign transactions or prescribed messages in the frontend.

2.2.3 Blockchain in P2P Energy Trading

In [7], it systematically analyzes the challenges that P2P energy trading may have, including lack of a reliable platform that can accurately manage the data generated from suppliers and consumers and record the transaction history in a tamper-proof and traceable way, requirement for availability and security for trading platforms against cyber-attacks etc.

Blockchain or distributed ledger technology provides feasible solutions to some challenges that P2P energy is facing. Blockchain's special architecture makes data it records counter-manipulation and anti-fraud. Blockchain further promotes decentralization in the smart grid to support prosumers' trading activities needless of intermediaries. With help of smart contract, blockchain can easily record transactions between two prosumers in a permissionless and transparent environment [8].

Blockchain offers two major functionalities in P2P energy trading. Both are all realized in the form of smart contract. One is acting as an auction market that takes ask and bid price from suppliers and consumers and match the suitable order. The other is payment settlement where the smart contract works with energy measuring devices such as smart meter to verify the energy transfer is performed in accordance with the agreement, then the payment can be settled via token based on the smart contract [9].

Regarding the procedure of P2P energy trading marketplace illustrated in section 2.1, how smart contract is participated is shown in Figure 5, prosumers' interactions with the smart contract and payment transactions are all recorded on blockchain.

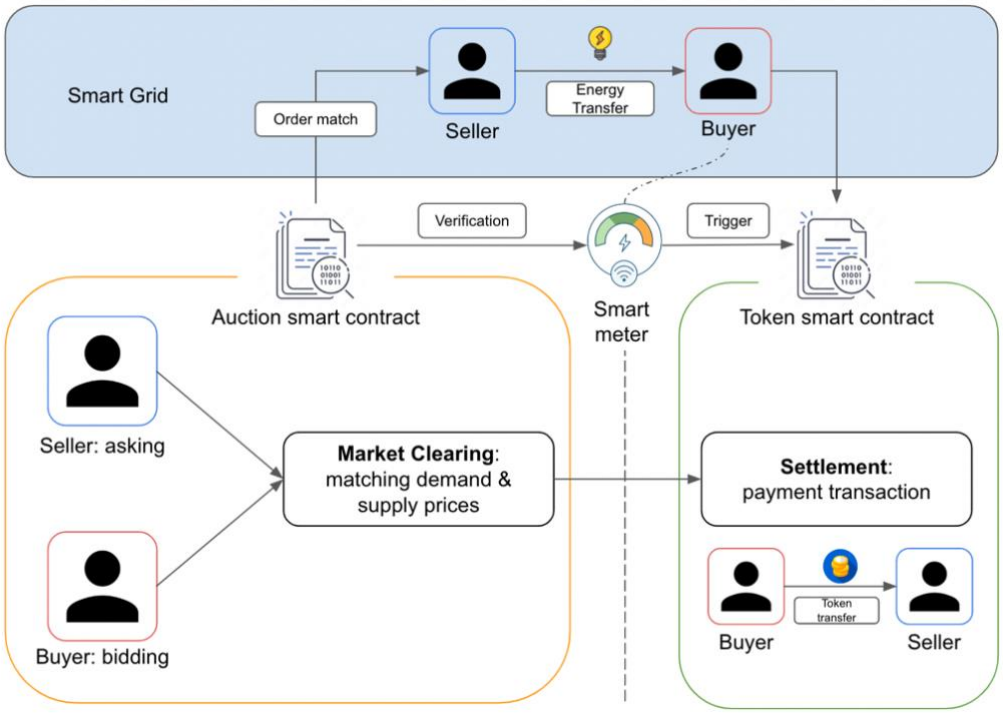

Figure 5 Smart contract in P2P energy trading

Other than the typical use case shown above, other utility cases of smart contract are also reviewed.

Reference [10] proposes an ERC20 token contract that digitizes the excess energy of prosumers. Buyers use this digitalized token to make payment. To guarantee the fairness during trading, the token can be locked up until the smart meter detects the actual delivery of energy. The locked token will then be released to pay suppliers.

Reference [11] proposes a decentralized exchange structure for the energy trading between virtual power plants (VPPs) and the power grid based on the concept of decentralized finance (DeFi). Energy surpluses from VPPs are tokenized as ERC20 token and the token can be swapped between VPPs using the decentralized exchange. The new owner of the swapped token then use it to redeem energy from the swapping counterparty VPP.

Besides research topics in P2P energy trading, examples of blockchain-based P2P energy trading network in real life are also reviewed. Reference [12] introduces the landscape of blockchain utilization in P2P energy trading. There are a total number of 189 companies working on blockchain in energy sector. Out of 50% of these projects utilize Ethereum blockchain to implement the trading network. Projects worth noticing include LO3 Energy, Power Ledger etc.

LO3 Energy is one of the earliest projects that leverage blockchain in the energy sector. It firstly launched P2P energy trading network in the Brooklyn microgrid in New York in 2015.

Power Ledger utilized Ethereum public blockchain and Solana-based consortium blockchain to build a multi-layer P2P energy trading system for prosumers to easily trade excessive energy to others. Its clients spread across South East Asia, North America, Australia etc.

Chapter 3 Methodology

This chapter discusses the major concept applied to the development of the relayer for the P2P energy trading based on blockchain.

As mentioned in section 1.3, the relayer focuses on the settlement stage in the P2P energy trading, where the typical design is that an ERC20 token is created as the payment method. Hence the transaction that the relayer handles is the transaction of the payment token from consumer to the supplier as shown in the section circled by the red dotted line in the Figure 6.

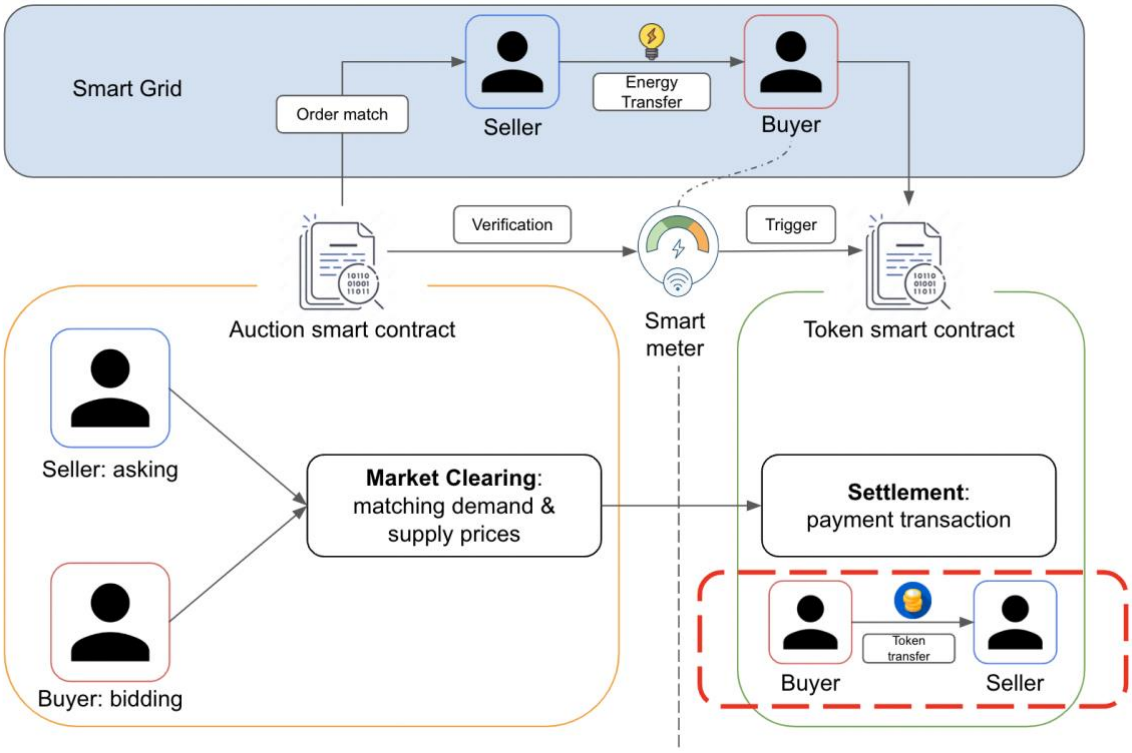

Figure 6 Relayer scope

3.1 ERC20 Standard

The ERC20 is a standard for Fungible Tokens on Ethereum blockchain. ERC20 standard makes sure that each token issued by the same ERC20 token contract share the same properties including type, value, symbol, name etc, which are the same to those of Bitcoin or Ether. The proposal is first introduced in 2015 by Fabian Vogelsteller and Vitalik Buterin.

ERC20 establishes a set of smart contract files in Solidity so that developers can import these files to issue their own ERC20 token. These prescribed files have been included in the OpenZeppelin library for Ethereum developers.

Next, we interpret some of the methods of ERC20 Solidity smart contract, which will be utilized or relevant to this project for better understanding.

3.1.1 BalanceOf()

```
function balanceOf(address account) external view returns (uint256);
```

Figure 7 ERC20 balanceOf() method

This method returns the amount of tokens owned by ‘account’ address. This method is a read-only method, since it does not cost any gas fee to call.

3.1.2 Transfer()

```
function transfer(address to, uint256 amount) external returns (bool);
```

Figure 8 ERC20 transfer() method

This method moves the ‘amount’ of tokens from the caller’s account to the ‘to’ account.

3.1.3 Approve() & allowance() & transferFrom()

Other than the direct transfer method by transfer(), ERC20 also introduces the indirect transfer method utilizing approve() + transferFrom().

```
function approve(address spender, uint256 amount) external returns (bool);
```

Figure 9 ERC20 approve() method

The approve() method can grant the 'amount' of allowance from the caller's account to the 'spender'account.

```
function allowance(address owner, address spender) external view returns (uint256);
```

Figure 10 ERC20 allowance() method

The allowance granted from the 'owner' account to the 'spender' account can be checked using the allowance() method. Since this is a read-only method, it does not cost any gas fee.

```
function transferFrom(
    address from,
    address to,
    uint256 amount
) external returns (bool);
```

Figure 11 ERC20 transferFrom() method

The transferFrom() method allows the spender to transfer the amount which is equal or smaller than the granted allowance from the owner account to the 'to' account.
Hence the approve() and transferFrom() combination allows the owner to give a certain amount of the token to a third party for operations such as transferring token to another account.

3.2 Meta-Transaction

Meta-Transactions are blockchain transactions that do not require users to pay the gas fee to initiate. Using web3 provider such as Metamask wallet, users sign the transaction request message on their computers' frontend and submits to the backend called 'relayer'. The relayer checks the validity of the signature and transaction message, that is to make sure the message is truly signed by the user instead of some pretenders. If validation passes, the relayer will submit the transaction and pay the gas fee for the user. Meta-Transaction enables users to interact with blockchain applications without the need of having cryptocurrency in their wallets, lowering the threshold for users' onboarding [13].

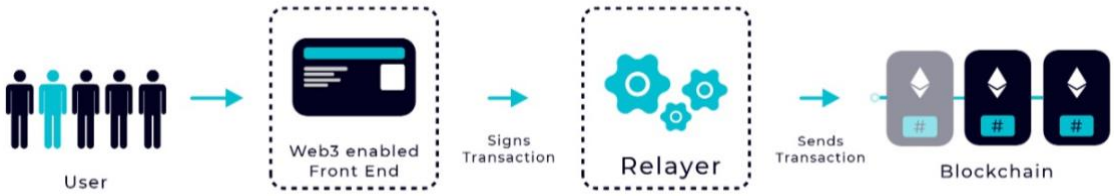

Figure 12 Meta-Transaction general procedure

To implement Meta-Transaction in the context of this project, that is achieving gasless ERC20 token transfer, two key components are required: the EIP712 standard and the ERC20 Permit() method.

3.3 EIP712 Standard

The full name of EIP712 is standard of typed structured data hashing and signing for Ethereum. It is a more advanced and secure method of signing transactions. Using this standard not only can a transaction be signed, and the signature be verified, but also data can be passed into the smart contract along with the signature. The signature can be

verified against that data to know if the signer is the one who sent the signature to be called in the transaction data.

EIP712 defines a standard structure for data and a defined process for generating hashes from structured messages. This hash is then used to generate a signature. In this way, signatures for different purposes can be constructed with distinction from other signatures.

EIP712 proposal includes:

- a specification of message structure for signing by web3 provider on the frontend
- a specification of data structure which is compatible with Solidity smart contract
- the hash algorithm with safety for preparation of signature generation and its optimized version of implementation in Ethereum Virtual Machine
- the mechanism of domain separator for signature distinction
- new RPC call eth_signTypedData for web3 providers to sign messages that are constructed based on EIP712

With these new concepts and implementations, EIP712 enables developers to construct the data structure that specifies certain method to be called in a smart contract. The data structure is input into the hash function and the output is signed by the user to tell the application that the user himself is willing to call this smart contract method. The method of smart contract has logic to verify that the submitted signature is indeed signed by the user and the method is activated. Hence EIP712 plays an important role in signature generation and signature verification in Meta-Transaction. The construction of a EIP712 message structure for signing on the frontend consists of two parts.

The first part is the message type including a structure of domain data for domain separator and a structure for message data to be signed on frontend. One thing to note is that the salt data type is an optional input rather than a necessary input. The verifying contract is the smart contract whose method requires signature verification to trigger. This method to be triggered also needs to follow the EIP712 standard, in this project's case is the ERC20 Permit method, which will be elaborated in section 3.4.

```
const domain = [
    { name: "name", type: "string" },
    { name: "version", type: "string" },
    { name: "chainId", type: "uint256" },
    { name: "verifyingContract", type: "address" },
    { name: "salt", type: "bytes32" },
];
const bid = [
    { name: "amount", type: "uint256" },
    { name: "bidder", type: "Identity" },
];
```

Figure 13 EIP712 message type configuration (example)

The second part is the actual input of the message structure defined above including the domain data and message data.

```
const domainData = {
    name: "Auction dApp",
    version: "2",
    chainId: parseInt(web3.version.network, 10),
    verifyingContract: "0x1C56346CD2A2Bf3202F771f50d3D14a367B48070",
    salt: "0xf2d857f4a3edcb9b78b4d503bfe733db1e3f6cdc2b7971ee739626c97e86a558"
};
var message = {
    amount: 100,
    bidder: {
        userId: 323,
        wallet: "0x3333333333333333333333333333333333333333"
    }
};
```

Figure 14 EIP712 message input (example)

The two parts combined together define a unique message for users to sign their transaction request on the frontend.

The signing method is eth_signTypedData, carried out in the web3 provider such as wallet.

Then the signed message is passed onto the verifying smart contract prescribed method. This method construct the hash of the message based on the defined message structure and the input based on EIP712. And using the input signature and the constructed hash, the smart contract is able to recover the address which signs this hash. This recover method is called ecrecover(), a prescribed function in Ethereum, specially built for ECDSA secp256k1 algorithm.

If this address is the same as the account who signs the message on the frontend, then it proves that the signature is not forged by someone else, and the prescribed operation in the smart contract can be implemented.

3.4 ERC20 Permit()

The Permit() method is developed by two decentralized finance projects MakerDAO and Uniswap. It is also referred to as EIP2612 standard that achieves Meta-Transaction in the ERC20 token transfer scenario.

Before the Permit() method, if the holder of an ERC20 token (prosumer) intends to transfer the token to a recipient (energy supplier), there are two ways to perform as elaborated in section 3.1:

- Direct transfer using transfer() method
- Indirect transfer using approve() + transferFrom() combination

In either way, the prosumer is required to pay the gas fee. However with the Permit() method, after verifying the signature input constructed using EIP712 standard, the prosumer enables a third party (relayer) to increase the prosumer's granted allowance to the relayer. After obtaining the allowance from prosumer, the relayer uses the transferFrom() method to help the prosumer transfer the amount of token to the intended supplier. The three ways of transferring are illustrated in Figure 15.

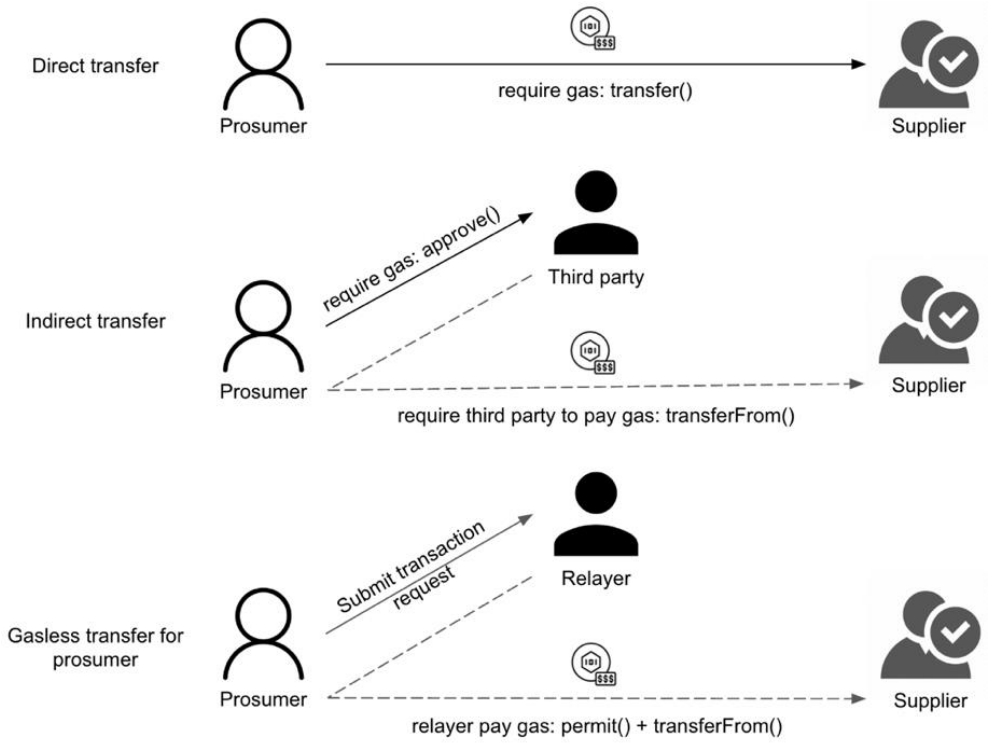

Figure 15 Three ways of making ERC20 token payment to energy supplier

Based on this idea, ERC20Permit, an alternation to the standard ERC20 smart contract, is developed by the Ethereum community. Developers can inherit the methods including permit() defined in this smart contract file to develop their own ERC20 token which is compatible with the Permit() method.

As shown in Figure 16, the permit() method requires input including owner, spender, amount, deadline, v, r, and s. That is to say the permit() method allows the prosumer (owner) to state that 'I would like to permit certain amount of allowance to the spender.

This request is valid until the deadline'. The v, r, and s together represent the signature generated based on this statement (request). The Permit() method verifies the authenticity of v, r, and s to prove that this request is indeed provided by the owner itself, then Permit() grants the 'amount' of 'allowance' from 'owner' to the 'spender'.

And regarding the _nonces[owner] variable, it is incremented by one each time the respective owner calls permit in this token contract. Its purpose is to prevent the replay attack where attackers reuse the past request and signature to call the permit() method.

```
function permit(
    address owner,
    address spender,
    uint256 amount,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) public virtual override {
    require(block.timestamp <= deadline, "ERC20Permit: expired deadline");


    bytes32 hashStruct = keccak256(
        abi.encode(
            _PERMIT_TYPEHASH,
            owner,
            spender,
            amount,
            _nonces[owner].current(),
            deadline
        )
    );

    bytes32 eip712DomainHash = _domainSeparator();

    bytes32 hash = keccak256(
        abi.encodePacked(uint16(0x1901), eip712DomainHash, hashStruct)
    );


    address signer = _recover(hash, v, r, s);

    require(signer == owner, "ERC20Permit: invalid signature");

    _nonces[owner].increment();
    _approve(owner, spender, amount);
}
```

Figure 16 ERC20 permit() method

3.5 Batching Mechanism

In the current blockchain-based P2P energy trading system, payment requests are submitted individually by users to blockchain during the settlement stage, as shown in Figure 17. Since these transactions are not likely to take place at the same time and they comes from different prosumers, payments are processed by different validators and are recorded into different blocks. This is an asynchronous process, it consumes many unnecessary validator resources, blockchain computation resources and much time.

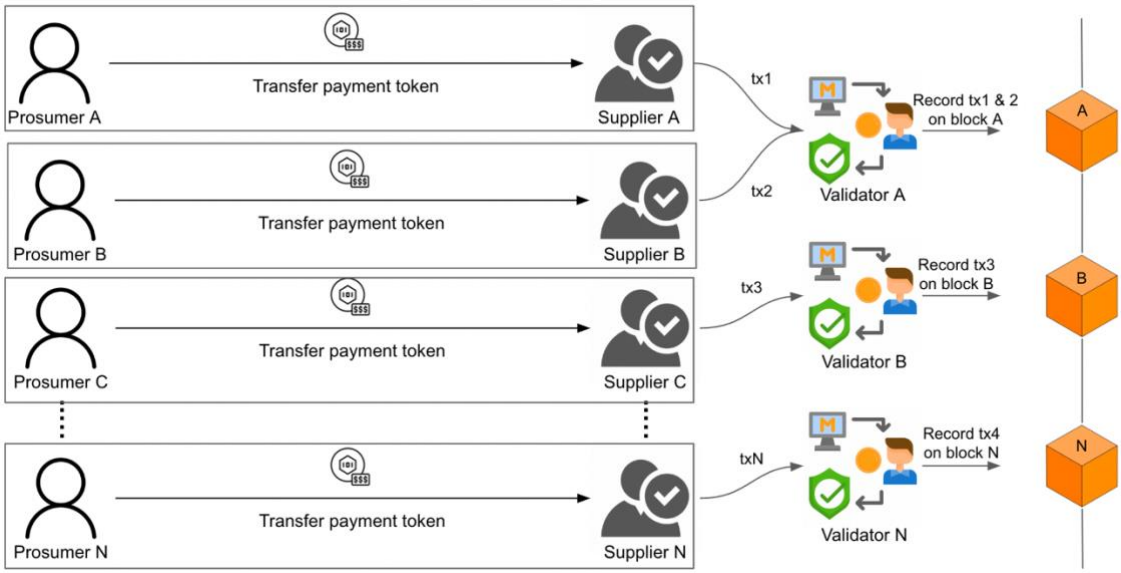

Figure 17 Payment without batching

What batching mechanism can do is bundling multiple payment requests into one transaction. In this way, the original multiple payment transactions submitted to blockchain and processed separately by the validators can be wrapped into one transaction so that these multiple payment requests can be processed by a single validator at the same time and then can be recorded into only one block of the blockchain, as shown in Figure 18.

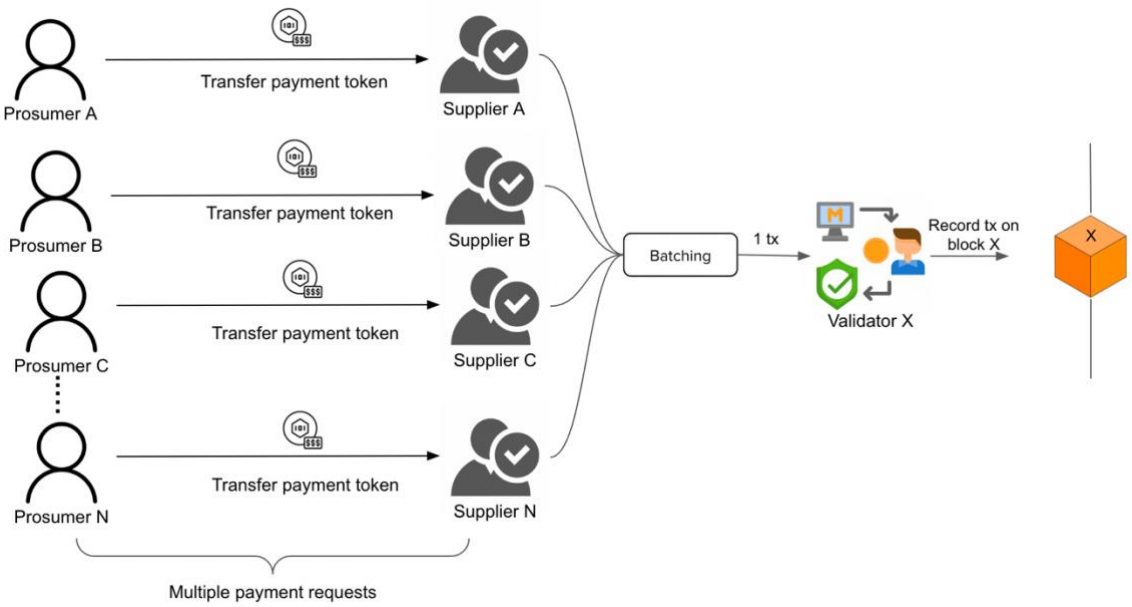

Figure 18 Payment with batching

This greatly reduces the burden of validators to process multiple payment requests in the settlement stage of a blockchain-based P2P energy trading system, and hence reduce congestion of the blockchain network no matter it is public or private blockchain. The trading efficiency is significantly improved. Validator resources and computation resources of blockchain are saved as well.

Chapter 4 Implementation

4.1 System Overview

This project built a full stack relayer service from frontend, backend, to smart contract. This section reviews the overall architecture and the workflow of the system.

4.1.1 System Architecture

In general, the relayer system consists of four components, as shown in Figure 19.

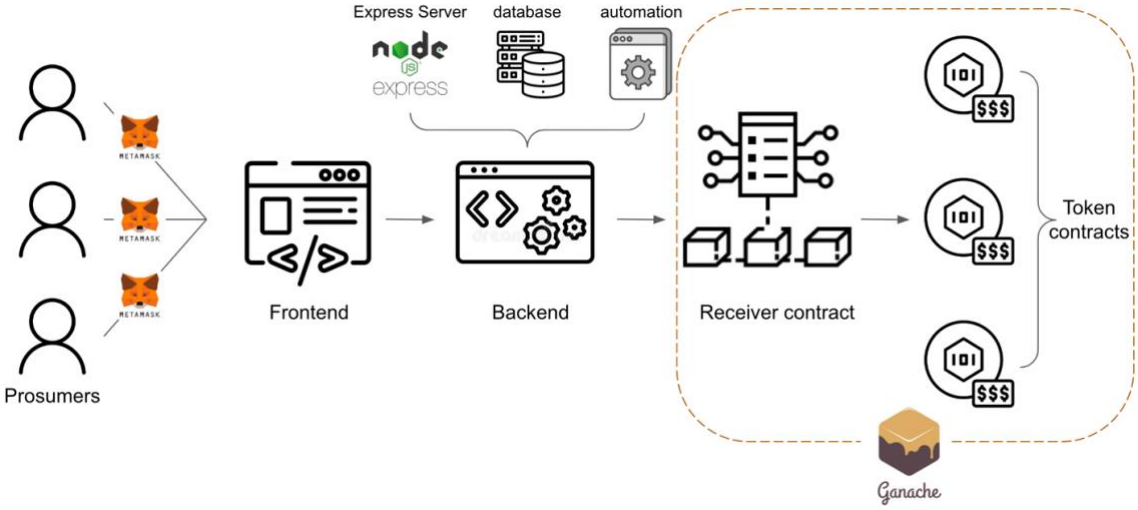

Figure 19 System architecture

- An interactive frontend web page where prosumers from the buyer side sign the payment transaction request through cryptocurrenty wallet Metamask. The request is constructed based on EIP712 standard. The frontend instantly verifies the signed request and inform the prosumer of the result on the web page. If it is valid, the payment request and the signature are transmitted to the relayer.

- The backend that receives the transmitted payment request and signature from the frontend. The backend consists of three components: Express server, database and an automated program that performs Meta-Transaction.

  - The Express server acts as a bridge between the relayer and the frontend. It double checks the validity of signature after receiving it and hands over to database. The Express server is a web application framework based on Node.js.
  - The database stores the transaction request and signature.
  - The automated program fetches transaction requests and signatures from the database, arranges and bundles them as a series of arrays as input to call the receiver contract to perform batched Meta-Transactions.

- The receiver contract that handles the input from the automated program from the backend and perform batched Meta-Transactions, which is a bunch of Meta-Transactions that enables multiple token payments to be settled in one time.

- The ERC20 token contract acts as payment token for prosumers. It is compatible with the EIP712 standard. The Payment token is not necessarily limited to only one. The receiver contract can handle multiple payment tokens transfers as long as they are compatible with the EIP712 standard.

A test environment is needed for smart contract deployment. Ganache private blockchain is utilized. It is a personal Ethereum blockchain.

4.1.2 System Workflow

The overall system workflow is shown in Figure 20:

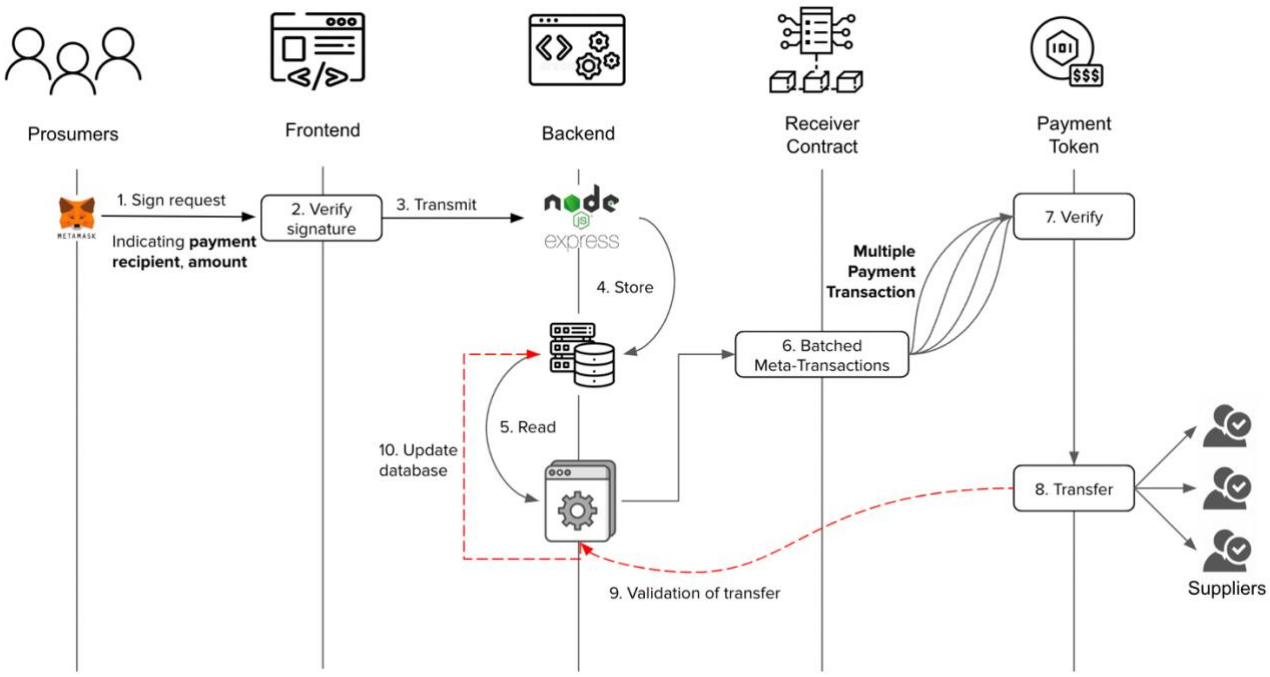

Figure 20 System workflow

1. Sign request: Prosumers input the payment information including the payment recipient's wallet address, the payment token amount in the frontend and use the Metamask wallet to sign the request.

2. The digital signature is generated, captured by the frontend. The frontend checks its validity and if it is valid, head towards step 3.

3. The frontend transmits the payment request and the signature together to the through an API service hosted by the Express server.

4. After receiving the payment request and signature, the Express server double checks the validity as frontend does. If it is valid, the server stores the request and signature into database.

5. The automated program fetches new payment requests and respective signatures from the database periodically, arranges and bundles them as a series of arrays as input to activate the receiver contract.

6. The receiver contract performs the batched Meta-Transactions once it is activated by the relayer automated program. Each Meta-Transaction leads to a follow-up activation on the respective payment token contract based on the ERC20 standard.

7. Regarding each Meta-Transaction, the corresponding ERC20 payment token contract verifies the validity of the signature.

8. The payment token can then be transferred to the respective supplier after the verification succeeds.

9. After transfers are finished, the automated program validates the result of payment transfer.

10. If validation passes, the automated program updates the status of the respective record of payment request.

4.2 Component Breakdown

The following sections look into the detailed implementation of each component and its workflow. They are illustrated according to the time order of development.

4.2.1 Smart Contract

Two contracts are involved. One is the payment token contract based on ERC20, the other is the receiver contract.

4.2.1.1 ERC20Permit Domain Separator

The ERC20Permit contract can be directly used since its development is already finished by the Ethereum community.

One thing needs to be altered is its domain separator so that the Permit() request generated is distinct from other permit() messages.

The update was performed in the _updateDominSeparator() method:

```
function _updateDomainSeparator() private returns (bytes32) {
    uint256 chainID = _chainID();

    // no need for assembly, running very rarely
    bytes32 newDomainSeparator = keccak256(
        abi.encode(
            keccak256(
                "EIP712Domain(string name,string version,uint256 chainId,address verifyingContract)"
            ),
            keccak256(bytes("FYP-A1068")), // Name
            keccak256(bytes("v1")),    // Version
            chainID,
            address(this)
        )
    );

    domainSeparators[chainID] = newDomainSeparator;

    return newDomainSeparator;
}
```

Figure 21 Update domain separator

The name was configured to “FYP-A1068”, and the version was configured to “v1”, as shown in Figure 21. The complete code can be viewed at https://github.com/BarrySauce/FYP-A1068/blob/main/contracts/ERC20Permit.sol

4.2.1.2 Token Contract

The token contract inherits ERC20Permit smart contract file. A new method named callPermit() was developed. This method can be viewed as a wrapped version of the permit() method, where callPermit() adds a require statement to guarantee that the caller of this method is restricted to the receiver contract itself.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.0;

import {ERC20, ERC20Permit} from "./ERC20Permit.sol";

contract TokenTemplate is ERC20Permit {

    address public receiver;

    constructor(address _receiver, address _holder, uint256 _initialSupply) ERC20("TokenName", "TokenSymbol") {
        receiver = _receiver;
        _mint(_holder, _initialSupply);
    }


    function callPermit(
        address _owner,
        address _spender,
        uint256 _amount,
        uint256 _deadline,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) external {
        require(msg.sender == receiver, "Only receiver is allowed to call this function");
        permit(_owner, _spender, _amount, _deadline, v, r, s);
    }
}
```

Figure 22 Token contract template

In the constructor() method, it initializes the receiver contract address, and mints the '_initialSupply' amount of token to the '_holder'.

The token total supply is restricted to 1000, all held by the '_holder' account.
To verify the relayer's compatibility with multiple ERC20 tokens, three payment tokens were coded according to the token template shown in Figure 22.

Their names are respectively EnergyTokenA (ETA), EnergyTokenB (ETB), EnergyTokenC (ETC). These three contract codes can be viewed at contracts folder: https://github.com/BarrySauce/FYP-A1068/tree/main/contracts.

4.2.1.3 Receiver Contract

The receiver contract can be viewed as a for loop program of Permit() + transferFrom() to implement the batching mechanism illustrated in section 3.5. The inputs are all arrays. Each item with the same index in these arrays forms a payment transaction request.

The receiver handles each payment transaction request in the respective array. First it calls the callPermit() method defined in our constructed token contract. After callPermit() is successful, the receiver contract receives the allowance granted from the owner. Then it will use the transferFrom() method to transfer the amount of respective token from the respective owner to the respective recipient.

Hence the batched payment requests are tackled by the receiver smart contract to implement actual payment token transfers, as shown in Figure 23.

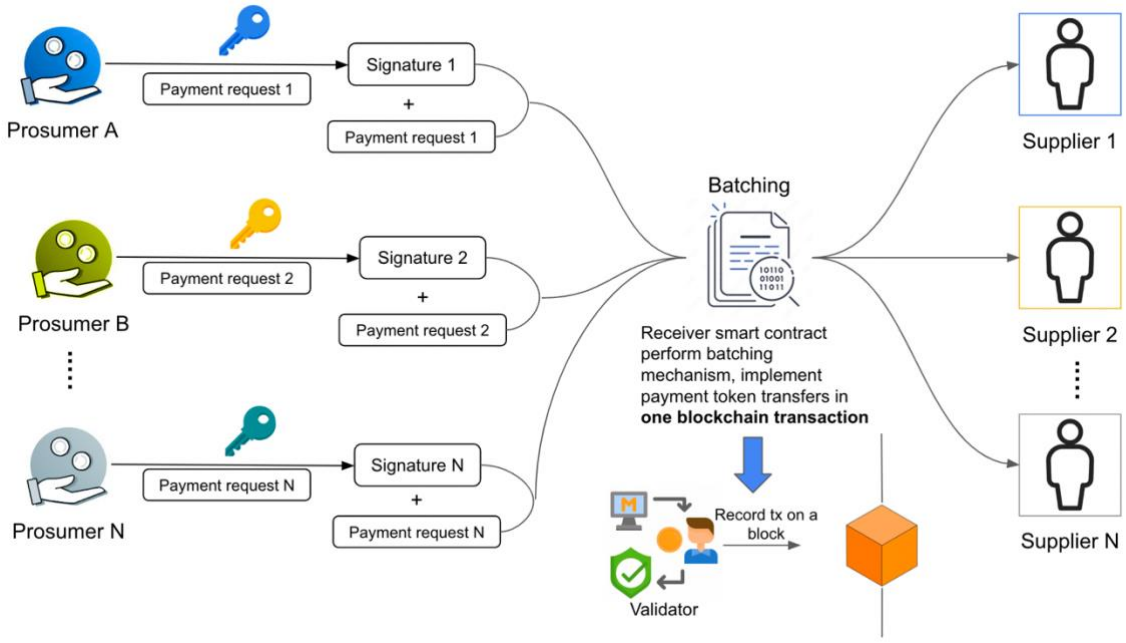

Figure 23 Receiver smart contract implement batching mechanism

Regarding the _noncesManger variable, it is used to prevent replay attack as mentioned in section 3.4.

The receiver contract code can be viewed at https://github.com/BarrySauce/FYP-A1068/blob/main/contracts/receiver.sol.

4.2.1.4 Deployment

Utilizing the smart contract development toolbox Hardhat. The contracts were compiled. The abstracts of the contracts were built which contains the contract ABI and bytecode.

The test environment adopted in this project is Ganache. Its user interface displays parameter including RPC server, Network ID etc, and test accounts with test Ether, as shown in Figure 24.

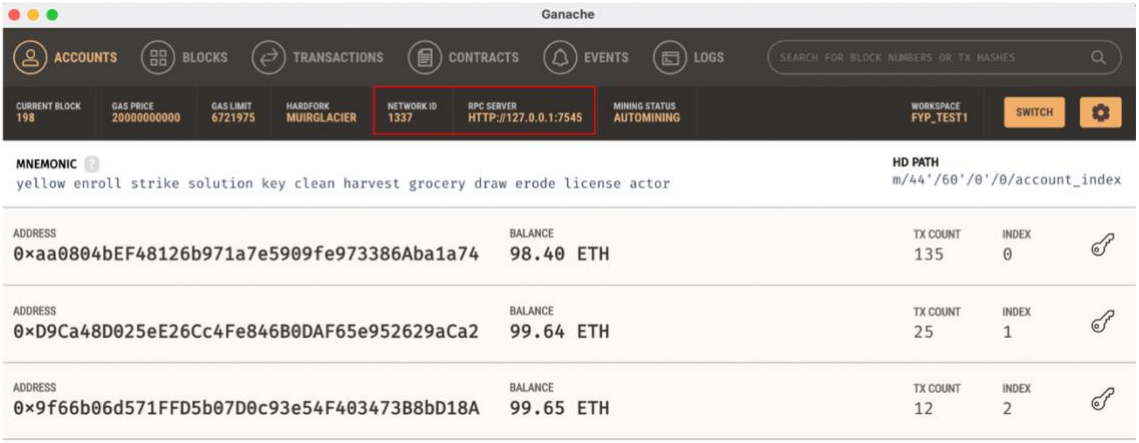

Figure 24 Ganache interface

Then configure the version of Solidity compiler to “0.8.0” and the network for deployment to ganache through the hardhat.config.js file (source: https://github.com/BarrySauce/FYP-A1068/blob/main/hardhat.config.js).
Utilizing javascript, an automated deploy script was written to deploy the receiver contract and the three payment token contracts Energy Token A, Energy Token B, Energy Token C on Ganache. The deploy script can be viewed at https://github.com/BarrySauce/FYP-A1068/blob/main/scripts/deploy.js.

Three test accounts with zero Ether in Ganache environment were adopted as the three initial holders for the respective three payment tokens.

Energy Token A: 0xD6B385c625CAfd2F4C57991f7df849Ede52eeE8f holds 1000

Energy Token B: 0x0D6DB60A12EA7bC554A41e7E99344F05CaDD7552 holds 1000

Energy Token C: 0x4F536A4d4D0b729fC6ed595383e3Bd5c54c2102F holds 1000

The deployed receiver contract and the three payment token contracts are shown in Figure 25:

```
Receiver contract deployed address is 0x296Cd92Fda484D62f992153172B5714E6E3A9B3c
ETA contract deployed address is 0x58d04b5Ca43EBEB1d66137c5C02011cf6411886A
ETB contract deployed address is 0x3C8ED0028587385b74140F502907Fd1030dFd10a
ETC contract deployed address is 0x0B6600B00e91eE1494257E1597A2ed8defb31dBc
```

Figure 25 Deployed contracts addresses

4.2.2 EIP712 Message

Next construct the EIP712 message in our case.

The first part is the message type including a structure of domain data for domain separator and a structure for message data. The structure of message data is in accordance with the Permit() method.

```
EIP712Domain:[
  {name:"name",type:"string"},
  {name:"version",type:"string"},
  {name:"chainId",type:"uint256"},
  {name:"verifyingContract",type:"address"}
],
Permit:[
{name:"owner", type:"address"},
{name:"spender",type:"address"},
{name:"value", type: "uint256"},
{name:"nonce", type: "uint256"},
{name:"deadline",type:"uint256"}
]
```

Figure 26 EIP712 message structure

The second part is the actual input of the message structure defined above including the domain data and message data. The name and version in domain are corresponding to the updated domain separator configuration in section 4.2.1.1. ChainId refers to the network ID of the blockchain environment. It is 1337 in accordance with Ganache. Verifying contract is the ERC20 permit token contract.

```
primaryType:"Permit",
domain:{name:"FYP-A1068",version:"v1",chainId:netId,verifyingContract: token},
message:{
  owner: signer,
  spender: relayer,
  value: amount,
  nonce: nonce_owner,
  deadline: deadline
}
})
```

Figure 27 EIP712 message input

4.2.3 Frontend

The frontend was built using React.js, an open-source JavaScript framework and library for building interactive user interface. The code can be viewed at https://github.com/BarrySauce/FYP-A1068/blob/main/frontend/src/App.js.

The frontend can be roughly split into three parts. One is the data acquisition part, where corresponding data are loading from Ganache blockchain and Metamask is connected to the web page. The second part is the import payment request message construction and signing function. And the third part is the interactive user interface displayed in the browser. The whole frontend in hosted by React.js on localhost:3000.

4.2.3.1 Data Acquisition

The coded **componentDidMount()** method fetches all the relevant data from the Ganache test blockchain as Table 1:

Table 1 Data Fetched from Ganache

Data name	Comment
accounts	a list of test accounts defined in Ganache
account	the current account (prosumer's wallet address) presented by Metamask
networkId	an unique number for the blockchain environment, in the case of Ganache, it is 1337
instance	the instance of the receiver contract created based on its contract ABI and address in Ganache

ETA_balance	current account's balance of Energy Token A
ETB_balance	current account's balance of Energy Token B
ETC_balance	current account's balance of Energy Token C

This method will start fetching relevant data from Ganache once it detects Metamask is running. If Metamask fails to run, this method will launch alert saying 'failed to load web3…' and gives out error messages in the console.

After data fetching is finished, this.setState() method is utilized to store all the needed global variables for other parts to use.

4.2.3.2 Payment Request Construction & Signing

This part collects the data defined by the EIP712 message elaborated in section 4.2.2 and use these data to construct the payment request. It enables the prosumer to sign the payment request through Metamask. The whole process is realized by the **signData()** method.

Table 2 shows the data applied to construct the payment request.

Table 2 Data for payment request construction

Data name	Data type	Comment
Domain data		
name	string	"FYP-1068", defined in section 4.2.1.1
version	string	'v1', defined in section 4.2.1.1
chainId	integer	same as the networkId in table 1, an unique number for blockchain environment. For Ganache, it is 1337
verifyingContract	Ethereum address	the respective ERC20 token contract whose permit() method is to be called
Message data		
owner	Ethereum address	the prosumer that makes payment

spender	Ethereum address	the third party that helps the owner to transfer the payment token to the supplier, which is the receiver contract elaborated in section 4.2.1.3
value	integer	amount of the payment token to be transferred
nonce	integer	nonce in case of replay attack
deadline	integer	payment request expires after this time

SignData() arrages the data in one payment request as Figure 28. Then by leveraging the eth_signTypedData method provided by the eth-sig-util dependency, signData() activates Metamask to sign the payment request.

After signing, the signature is also transmitted to signData() method which will then perform verification to check whether the signature is signed by the indicated prosumer.

```
const msgParams = JSON.stringify({
  types:
  {
  EIP712Domain:[
    {name:"name",type:"string"},
    {name:"version",type:"string"},
    {name:"chainId",type:"uint256"},
    {name:"verifyingContract",type:"address"}
  ],
  Permit:[
  {name:"owner", type:"address"},
  {name:"spender",type:"address"},
  {name:"value", type: "uint256"},
  {name:"nonce", type: "uint256"},
  {name:"deadline",type:"uint256"}
  ]
},

primaryType:"Permit",
domain:{name:"FYP-A1068",version:"v1",chainId:netId,verifyingContract: token},
message:{
  owner: signer,
  spender: relayer,
  value: amount,
  nonce: nonce_owner,
  deadline: deadline
}
})
```

Figure 28 Complete payment request

4.2.3.3 User Interface

The user interface is for prosumer to input the payment details including the payment token to use, the amount of payment token to transfer and the wallet address of the supplier, as shown in Figure 29. Then the prosumer activates Metamask to sign the payment request constructed based on the inputs by clicking the ‘Submit’ button.

The signature is produced and inputs into the signData() method which then performs verification on the signature. If the signature is valid, a pop-up window will show up on the frontend to inform the prosumer that the signature is valid, otherwise a pop-up window alerting the verification fails will show up on the frontend.

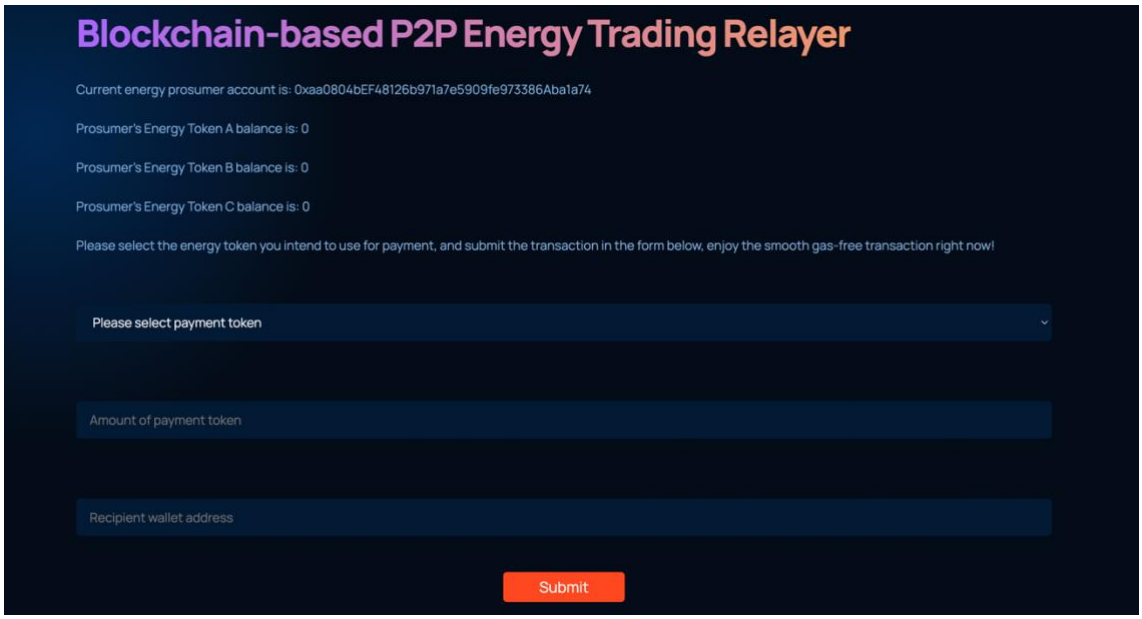

Figure 29 User Interface

4.2.4 Backend

The backend consists of an Express server supported by node.js, a MySQL database and an automated program. They work together to bundle the payment request stored in the database and input in the receiver contract for payment transaction.

4.2.4.1 Express Server

The Express server acts as a bridge between the frontend and the database. By utilizing the HTTP Post method, we created an API for callers to transmit data towards the backend.

The main file of Express server can be viewed at https://github.com/BarrySauce/FYP-A1068/blob/main/server/index.js. The server is hosted on localhost:4001.

The API based on HTTP Post method offers 2 functionalities:

1. It enables the frontend to transmit the payment request and the respective signature to the backend
2. After receiving the payment request and the signature, it double checks the validity of the signature. If the signature is valid, the API stores the payment request and the signature into MySQL database.

The API is hosted on localhost:4001/txes, its code can be viewed https://github.com/BarrySauce/FYP-A1068/blob/main/server/routes/txes.js.

4.2.4.2 Connect Express Server with Frontend

To connect the Frontend with the Express server, the signData() method needs alternation.

After signData() activates Metamask on the frontend to sign the payment request and the produced signature reaches to the frontend, the frontend checks the validity of the signature, if the signature is valid, signData() will then call the API service (HTTP Post) at localhost:4001/txes to transmit the payment request and the signature to the backend.

```
const requestOptions = {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({
        TokenAddress: token,
        SignerAddress: signer,
        spender: relayer,
        amount: amount,
        deadline: deadline,
        RecipientAddress: recipient,
        sig: sig,
        msgParams: msgParams,
        ExecutionStatus: status
    })
};

if (ethUtil.toChecksumAddress(recovered) === ethUtil.toChecksumAddress(from)) {

  fetch("http://localhost:4001/txes", requestOptions)
  .then(response => response.json())
  .then(result => {
      console.log(result);
  })

  alert('Successfully ecRecovered signer as ' + from)
} else {
  alert('Failed to verify signer when comparing ' + result + ' to ' + from)
}
```

Figure 30 Frontend calls backend's API to transmit data

The transmitted data structure is shown in Table 3:

Table 3 Data structure for transmitted payment request

Data name	Data type	Comment
TokenAddress	Ethereum address	payment token address selected by the prosumer
SignerAddress	Ethereum address	prosumer's wallet address
spender	Ethereum address	the receiver smart contract's address
amount	integer	number of payment token to be transferred

deadline	integer	payment request expires after the deadline
RecipientAddress	Ethereum address	the supplier's wallet address to receive payment
sig	string	the signature generated by Metamask signing the payment request
msgParams	multiple type	as shown in Figure 28, a complete payment request. This data is used for backend to double validate the signature
ExecutionStatus	string	initial value is 'default'. This status will be later be modified by the automated program in the database to reflect whether the request is successfully handled by the relayer

4.2.4.3 MySQL Database

MySQL database stores the complete validated payment request passed by the Express server.

To connect the database with the Express server, a module called Sequelize is leveraged. It not only enables the whole backend to connect with the database, but also offers the ability of creating or altering tables without using database's specified programming language such as SQL.

To visualize the data table, a graphic user interface called MySQL Workbench was utilized.

First a SQL instance was created via MySQL Workbench, username and password are required to set up. This instance is hosted on localhost:3306, as shown in Figure 31.

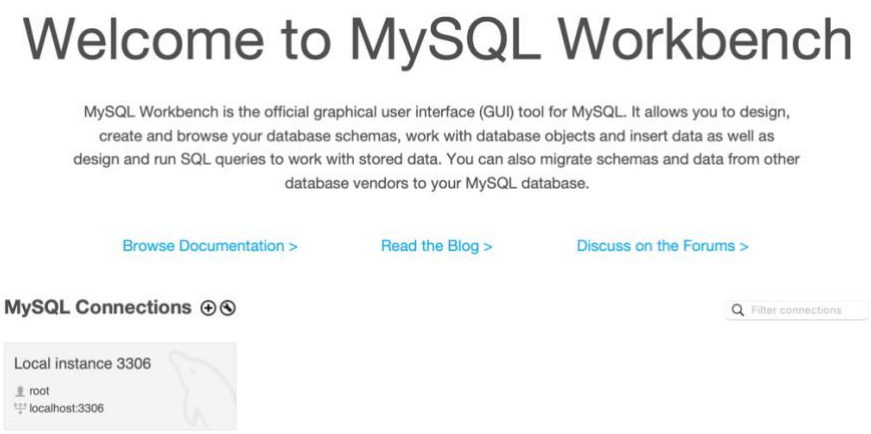

Figure 31 Local instance on MySQL Workbench

Then a database named A1068 which contains multiple data tables was created in the instance. Next, the Sequelize module was installed. To connect the database to the Express server, the config.js file (https://github.com/BarrySauce/FYP-A1068/blob/main/server/config/config.json) was modified. In this file, the development array items were filled in with the respective parameter from the local instance.

```
"development": {
  "username": "Local instance username",
  "password": "local instance password",
  "database": "A1068",
  "host": "127.0.0.1",
  "dialect": "mysql"
},
```

Figure 32 Configuration in Sequelize config file

Then a data table was created. This table records the transmitted payment request & signature. Its data fields align with the data structure indicated at Table 3. The data table name is Txes (https://github.com/BarrySauce/FYP-A1068/blob/main/server/models/Txes.js).

```
module.exports = (sequelize, DataTypes) => {
  const Txes = sequelize.define("Txes", {
    TokenAddress: {
      type: DataTypes.STRING(100),
    },
    SignerAddress: {
      type: DataTypes.STRING(100),
    },
    spender: {
      type: DataTypes.STRING(100),
    },
    amount: {
      type: DataTypes.INTEGER,
    },
    deadline: {
      type: DataTypes.BIGINT,
    },
    RecipientAddress: {
      type: DataTypes.STRING(100)
    },
    sig: {
      type: DataTypes.STRING,
    },
    msgParams: {
      type: DataTypes.JSON,
    },
    ExecutionStatus: {
      type: DataTypes.STRING,
    }
  });

  return Txes;
};
```

Figure 33 Data table Txes defined by sequelize

After running the command ‘npm run start’, an empty table with the intended data fields was created in the A1068 database, as shown in Figure 34.

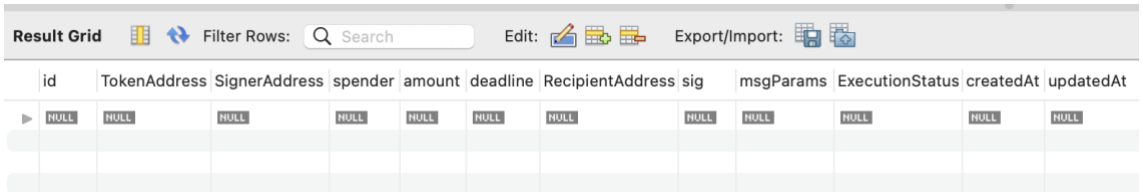

Figure 34 Empty Data table Txes defined by sequelize

After the connection and initialization, the Express server can write the transmitted payment request from frontend in the Txes data table.

An example data table filled with several payment requests is shown in Figure 35.

id	TokenAddress	SignerAddress	spender	amount	deadline	RecipientAddress	sig	msgParams	ExecutionStatus
1	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0xD6B385c625CAfd2F4C579917fd849Ede52e...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680175864	0xaa0804bEF48126b971a7e5909fe973386Aba...	0x295c569da23d871e72799c72c1ae6ce9ed3b...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful
2	0x3CBED0028587385b74140F502907Fd103dd...	0x03DB80A12EA7bC554A41e7E99344F05Ca...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680175917	0xD9Ca48D025eE26Co4Fe846B0DAF65e9526...	0xda54387784d202edcc257b70e8afd5292f69a...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful
3	0x0B6600B00e91eE1494257E1597A2edBdefb3...	0x4F536A4d4D0b729fC6ed595383e38d5c54c2...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680175958	0x9f66b06d571FFD5b07D0c93e54F4034738b...	0x445c0Daa857ae5f27083cd7fe3a1788e022847...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful
4	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0xaa0804bEF48126b971a7e5909fe973386Aba...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680176496	0xD6B385c625CAfd2F4C57991f7df849Ede52e...	0x8c695db03ce27a08b1eb4738fbac007b665c...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful
5	0x3CBED0028587385b74140F502907Fd103dd...	0xD9Ca48D025eE26Co4Fe846B0DAF65e9526...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680176575	0xDD6DB80A12EA7bC554A41e7E99344F05Ca...	0x941c68b121cb04690643bac1262d716770244f...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful
6	0x0B6600B00e91eE1494257E1597A2edBdefb3...	0x9f66b06d571FFD5b07D0c93e54F4034738bb...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680176640	0x4F536A4d4D0b729fC6ed595383e38d5c54c2...	0x5e918f91265ba5c08703669a04d2bd9d132...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful
7	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0x9f66b06d571FFD5b07D0c93e54F4034738bb...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680176673	0x4F536A4d4D0b729fC6ed595383e38d5c54c2...	0xb2b6694a2eae23396e600889dc3bdf4185c3d...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	failed
8	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0x9f66b06d571FFD5b07D0c93e54F4034738bb...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680176887	0xaa0804bEF48126b971a7e5909fe973386Aba...	0x2a7a6044ebb191858940b84794acf8c3387ea...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	failed
9	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0xD6B385c625CAfd2F4C579917fd849Ede52e...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680177139	0xaa0804bEF48126b971a7e5909fe973386Aba...	0xecc6f7098c4b6575c1a44868f8b3641cf5af362...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful
10	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0xaa0804bEF48126b971a7e5909fe973386Aba...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680177206	0xD6B385c625CAfd2F4C57991f7df849Ede52e...	0x35bd386e474ba0b076bc3cb982b2d44b87f5...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	failed
11	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0xD6B385c625CAfd2F4C579917fd849Ede52e...	0x296Cd92Fda484D62f992153172B5714E6E3...	4	1680177418	0xaa0804bEF48126b971a7e5909fe973386Aba...	0x0ac66199d6c28c7abb4d81ed42e9b54cfe85a...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful
12	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0xD6B385c625CAfd2F4C579917fd849Ede52e...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680177559	0xaa0804bEF48126b971a7e5909fe973386Aba...	0xc4381d475d9b0ae111373427ba17482c02abe...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	failed
13	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0xaa0804bEF48126b971a7e5909fe973386Aba...	0x296Cd92Fda484D62f992153172B5714E6E3...	4	1680177598	0xD6B385c625CAfd2F4C57991f7df849Ede52e...	0x1155ca37e7de6d6092b6405464fb49ad033821...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	failed
14	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0xD6B385c625CAfd2F4C579917fd849Ede52e...	0x296Cd92Fda484D62f992153172B5714E6E3...	5	1680177923	0xaa0804bEF48126b971a7e5909fe973386Aba...	0x7685a3fcce42ffb683a8b4b162575ed24bb09...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful
15	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0xD6B385c625CAfd2F4C579917fd849Ede52e...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680178093	0xaa0804bEF48126b971a7e5909fe973386Aba...	0x87b518399ad818518b1979c1dcf88a85606eb...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful
16	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0xaa0804bEF48126b971a7e5909fe973386Aba...	0x296Cd92Fda484D62f992153172B5714E6E3...	5	1680178124	0xD6B385c625CAfd2F4C57991f7df849Ede52e...	0x1b0ab36343615d8eaf4ae8e8d8b64f1fb4b90...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful
17	0x58d04b5Ca43EBEB1d96137c5C02011cf6411...	0xD6B385c625CAfd2F4C579917fd849Ede52e...	0x296Cd92Fda484D62f992153172B5714E6E3...	1	1680196513	0xaa0804bEF48126b971a7e5909fe973386Aba...	0x81916320a21a338bf0aec03046aa0c2075b547...	{"types\":{"EIP712Domain\":[{"name\":"name\"...	successful

Figure 35 Data table Txes with records

4.2.4.4 Automated Program

The automated program works as a caller to activate the callPermit() method in the receiver contract to initiate bundled Meta_Transactions. It is written in JavaScript as well, also hosted by the Express server.

The automated program reads the payment request from the Txes data table to detect whether newly unhandled payment requests are inserted in. The detection is done based on the ExecutionStatus data field of the payment request. Its initial value is ‘default’, and will be altered to ‘successful’ or ‘failed’ by the automated program. Hence as long as the payment request with ExecutionStatus being ‘default’ is found, it means newly unhandled payment request is transmitted from the frontend via the Express Server. Once detected, the program arranges the payment requests into arrays according to the data fields required by the receiver contract activatePermit() method inputs.

```
function activatePermit(
    address[] memory _tokenContracts,
    uint256[] memory _amounts,
    address[] memory _owners,
    uint256[] memory _deadlines,
    uint8[] memory v,
    bytes32[] memory r,
    bytes32[] memory s,
    address[] memory _recipients
)
```

Figure 36 Receiver contract activatePermit()

Hence the corresponding arrays shown in Figure 37 were created by the automated program according to Figure 36.

```
const tokenContracts = [];
const amounts = [];
const owners = [];
const deadlines = [];
const vArray = [];
const rArray = [];
const sArray = [];
const recipients = [];
```

Figure 37 arrays created by the automated program

The v, r, s are different segmentations extracted from the signature. The segmentation is necessary to accommodate the input defined by the Permit() method from ERC20.

In the next step, the program filters the feasible payment request and appends each of its data item to the corresponding array. By feasibility, we mean the prosumer (token owner) has the enough balance to pay the amount of token to the supplier (recipient). The payment requests without sufficient balance will not be appended into the array. Their ExecutionStatus recorded in the database will be altered to ‘failed’.

After appending is finished, the program activates the activatePermit() method in the receiver contract, using the appended arrays as inputs. To achieve automation of activating transaction, the private key of a wallet account with sufficient balance of Ether must be imported into the program. Then this private key is used to sign the activatePermit() transaction, which is the batched Meta-Transactions, and pay the gas fee.

```
//execute activePermit transaction
const activatePermitQuery = contractReceiver.methods.activatePermit(
  tokenContracts,
  amounts,
  owners,
  deadlines,
  vArray,
  rArray,
  sArray,
  recipients
  );

const encodedABI = activatePermitQuery.encodeABI();

const tx = {
  from:botAddress,
  to: receiverAddress,
  gas: 2100000,
  data: encodedABI,
};

const signed  = await web3.eth.accounts.signTransaction(tx, privatekey);
```

Figure 38 Call the activatePermit() method using imported private key

After activatePermit() transaction is done, the final step is to update the ExecutionStatus of the payment requests which are appended to the array. Their ExecutionStatus values are altered from ‘default’ to ‘successful’. The implementation of this program can be viewed at https://github.com/BarrySauce/FYP-A1068/blob/main/server/relay.js.

And of course this program cannot run only once, since new payment requests are constantly being appended to the database. The automated program needs to perform its logic after every time interval. In the framework of node.js, this can be achieved via the node-cron module which enables a program to run regularly. The time interval can be configured.

After installing this module, a cron.js file (https://github.com/BarrySauce/FYP-A1068/blob/main/server/cron.js) was created. The program is imported to this cron.js file and cron was configured to run the automated program every minute.

4.3 Performance Review

This section reviews the performance of the relayer system to better understand its advantage and potential risks. Section 4.3.1 ~ 4.3.2 covers the performance review of the system based on private blockchain. Section 4.3.3 covers the performance review of the system based on public blockchain.

4.3.1 Efficiency

With batching and submitting multiple payments in one single transaction, the number of one-on-one payment transactions between prosumers and suppliers are reduced. With less interaction with the blockchain, the less network congestion there will be, especially considering the scenario where the blockchain-based P2P energy trading network achieves wide adoption among households and communities. The decrease in payment transactions will be considerable, saving a lot of validator resources and computation resources of blockchain, as shown in Figure 39.

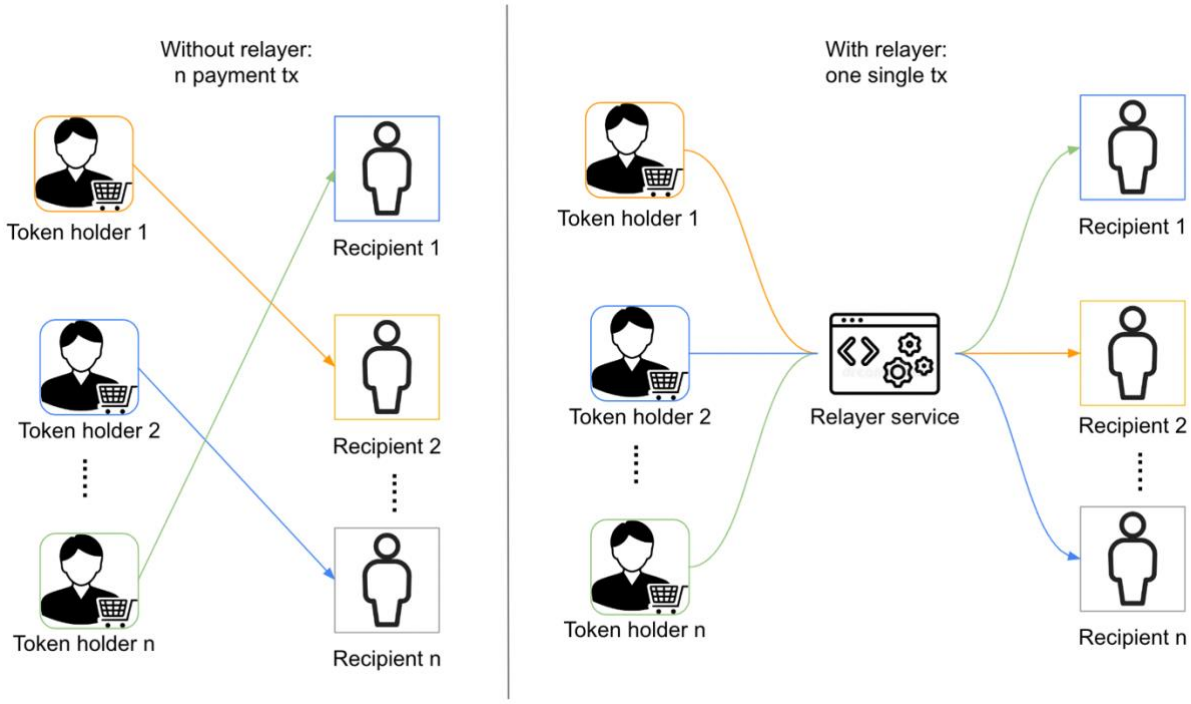

Figure 39 Number of payment transactions with/without relayer

In terms of time efficiency, currently the relayer runs its program for every 1 minute, considering the transaction processing and confirmation time on Ethereum is about 30 seconds ~ 1 minutes. The prosumer needs to wait approximately at most 2 minutes for its payment request being handled.

4.3.2 Cost

This section compares the gas cost of the relayer service. This gas cost is all paid by the relayer itself.

4.3.2.1 Relayer vs transfer()

As mentioned in the previous sections, transfer() is the direct transfer method in which the prosumer pays gas to transfer his own payment token to the supplier. A simple experiment is carried out in the Ganache environment:

Three newly unhandled payment requests are inserted into the database via frontend. Each request is transferring one different Energy Token (ETA, ETB, ETC) to different recipient, as shown in Figure 40.

id	TokenAddress	SignerAddress	spender	amount
1	0x58d04b5Ca43EBEB1d66137c5C02011cf6411...	0xD6B385c625CAfd2F4C57991f7df849Ede52e...	0x296Cd92Fda484D62f992153172B5714E6E3...	1
2	0x3C8ED0028587385b74140F502907Fd1030d...	0x0D6DB60A12EA7bC554A41e7E99344F05Ca...	0x296Cd92Fda484D62f992153172B5714E6E3...	1
3	0x0B6600B00e91eE1494257E1597A2ed8defb3...	0x4F536A4d4D0b729fC6ed595383e3Bd5c54c2...	0x296Cd92Fda484D62f992153172B5714E6E3...	1
NULL	NULL	NULL	NULL	NULL

Figure 40 Three newly added payment requests

The relayer detects the new requests, batches them and launches them via the receiver contract. The gas used is 236601, as shown in Figure 41.

Figure 41 Batched Meta-Transaction detail

Then let three recipients in the relayer's scenario to transfer the token back to the owner. Each recipient has a certain amount of test Ether with them, the transaction records are shown in Figure 42.

Figure 42 Three transfer() transactions

Each transfer() transaction costs 21767 gas. The three combined together is 65301 gas. By comparison, the relayer raises the gas cost by 70% in three payment requests scenario. This is understandable since the relayer's smart contracts receiver contract adopts for loop to submit multiple transactions. The gas cost increases due to the amount of calculation and the increasing complexity of codes.

4.3.2.2 Relayer vs approve() + transferFrom()

We used the relayer to transfer five amount of Energy Token A to the recipient who has certain amount of test Ether. Then the recipient used approve() to grant another user one allowance of Energy Token A. This user then called the transferFrom() method to transfer this one Energy Token A. The details of these two transactions are shown in Figure 43:

Figure 43 Transaction detail of the approve() + transferFrom()

The total gas cost of approve() + transferFrom() combination is 44757 + 30819 = 75576 The average gas cost of three payment requests handled by the relayer is 236601/3 = 78876. The cost is very close.

Hence in terms of the cost, the relayer service is more expensive than the transfer() method, and is very close to the combination of approve() + transferFrom(). Another interesting thing observed is that with the increasing number of payment requests being handled in one batched Meta-Transaction, the total gas fee increases, but the average gas fee for each request decreases. For instance, the batched Meta-Transaction with only one payment request vs the batched Meta-Transaction with three payment requests. The former one costs 97356 gas. The later one costs on the average 78876 for one request.

Figure 44 Gas cost of batched-Meta Transaction with only one payment request

4.3.3 Performance on Public Blockchain

Further experiments that evaluate the relayer service deployed on popular public blockchain testnets including Ethereum Goerli testnet, Polygon PoS Mumbai testnet, Arbitrum Goerli testnet and Optimism Goerli testnet were conducted. Efficiency and cost are still the two major focus of the experiments.

First the receiver contract and the three payment tokens were deployed on the respective testnet. Then the time and gas fee required by one Meta-Transaction that batched 3 payment requests and 6 payment requests were measured respectively. Considering the public blockchain testnet is a dynamic environment, the measurement was performed for multiple times to acquire the average data of the time and gas fee required. The results are displayed in the figure below. Moreover, since the testnets adopt different native tokens, so gas fee is measured in USD.

Efficiency (measured in second) of Meta-Transaction that batched 3 payment requests and 6 payment request on different testnet are shown in Figure 45.

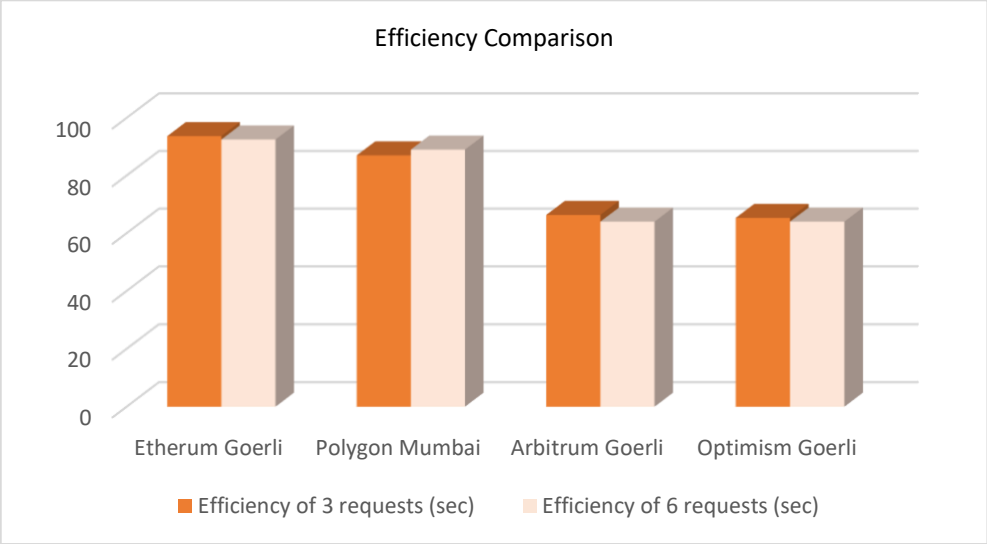

Figure 45 Comparison of relayer efficiency across different public testnet

Cost (measured in USD) of Meta-Transaction that batched 3 payment requests and 6 payment request on different testnet are shown in Figure 46.

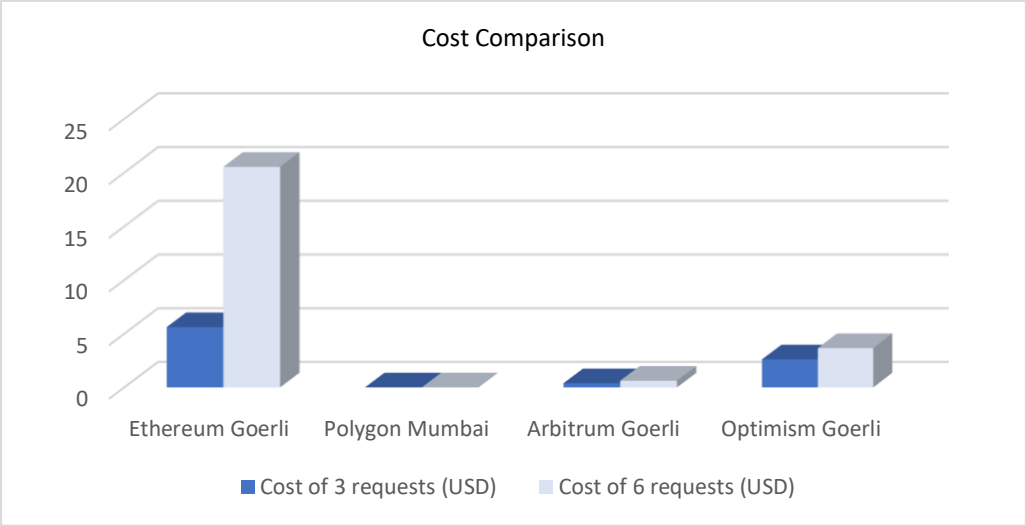

Figure 46 Comparison of relayer cost across different public testnet

By comparison, it can be observed that Arbitrum and Optimism offered the highest efficiency. Polygon offers the lowest cost. This is expected as Arbitrum and Optimism are both Layer 2 solution of blockchain that greatly improves the scalability of blockchain. And Polygon native token Matic has the lowest price, and the gas it costed was low as well, hence it is the most economic option among these public blockchains. Ethereum is the least time-efficient and cost-efficient option among the four.

Another observation is that the gas fee is not proportional to the number of batched payment requests. The average cost of one payment request in 6 batched requests is lower than that of 3 batched requests for Polygon, Arbitrum and Optimism, which validates the observation stated in section 4.3.2.2. Ethereum presents the opposite result. 6 batched requests costed 20.54 USD, while 3 batched requests costed 5.62 USD.

For the specific data of the experiment, please refer to Appendix A.2 for more details.

Chapter 5 Conclusion and Future Work

5.1 Conclusion

Peer-to-peer energy trading greatly improves the utilization of energy and efficiency of energy trading. With the help of blockchain and smart contract, an open and permissionless market can be constructed on a secure, transparent and distributed ledger. Transactions are recorded with attributes including traceability, verifiability and non-repudiation, which enhance trust among prosumers.

However due to the nature of blockchain, everything currently is settled in the crypto-native method, which leads to isolation between traditional industries and the web3 world. In the energy trading process based on blockchain, prosumers must pay gas fee to launch their payment transactions to suppliers in the final settlement stage. This causes great inconvenience for people who are not very familiar with blockchain to get onboard on the platform. This issue must be tackled otherwise it can be foreseen that blockchain-based p2p energy trading platform will not be adopted by the greater public.

Utilizing the web3-native EIP712 standard, this project built a full stack relayer that can perform payment transaction for the prosumer. This eliminates the need for prosumers to pay gas during the settlement stage in the blockchain-based P2P energy market. Prosumers can input their payment requests via the straightforward user interface, and wait for at around 2 minutes before the payment transaction is settled by the relayer.

Moreover, the relayer service also improves the efficiency of settlement. With batching and submitting multiple transactions in one transaction at a time, the number of one-on-one payment transactions between prosumers and suppliers is reduced. Blockchain network congestion can be mitigated to some extent, unnecessary cost is reduced.

5.2 Future Works

Future works may include enhancement on the security by further decentralizing the system architecture, particularly focusing on the decentralization of database. Decentralized storage protocol such as IPFS or Swarm can be considered to replace the centralized MySQL database. A trade-off needs to be taken into account since most the performance of decentralized storage is not as fast as the centralized database.

Another aspect for decentralization is the relayer service itself. Multiple relayers can be deployed to form a decentralized relayer network. When a prosumer submits a payment request to the relayer network, there is a selection process that randomly picks a relayer to launch the payment transaction for the prosumer. With multiple relayers being available, the risk of single point failure can be reduced.

Further discussions including introducing economic model to the relayer service such as Fiat payment gateway, adoption in other aspects of P2P energy trading such as energy tokenization can be carried out. The future of P2P energy trading and blockchain is limitless, and countless possibilities are still waiting for us to discover.

Reflection on Learning Outcome Attainment

There are three major aspects I would like to reflect on what I have learnt and achieved.

First is ***Problem Analysis***. I did literature review on implementation and performance analysis of P2P energy trading based on blockchain to try to accurately identify and analyse the existing issues of blockchain-based P2P energy trading that hold it back from great adoption. Two major issues were figured out, which are efficiency and usability. Then based on these two issues, I further studied the architecture limitation of blockchain and transactions to supplement my problem statement.

Second is ***Engineering Knowledge***. Through this project, I got the opportunity to learn multiple programming and software engineering concepts, techniques and knowledge including general programming such as Node.js backend, SQL database, and specific programming in blockchain such as EIP712 standard, digital signature, realization of Meta-Transaction etc. Moreover, I learnt how to constantly update and manage my code repository to make my code neat and easy to read and change. Utilizing these techniques, I successfully built the relayer service.

Third is ***Communication***. I synchronized with my supervisor Prof. Gooi and my co-supervisor Dr. Yang when I finished a certain milestone of my project. These conversations were all very meaningful and constructive for me to figure out what aspects were required for correction and what steps were supposed to take in the next section. I am truly thankful for the advice and support they offered to me. I could not accomplish this project without their valuable guidance.

References

[1] Y. Zhou, J. Wu, C. Long, and W. Ming, “State-of-the-Art Analysis and Perspectives for Peer-to-Peer Energy Trading,” Engineering, vol. 6, no. 7, pp. 739–753, Jul. 2020, doi: https://doi.org/10.1016/j.eng.2020.06.002.

[2] C. Zhang, J. Wu, Y. Zhou, M. Cheng, and C. Long, “Peer-to-Peer energy trading in a Microgrid,” Applied Energy, vol. 220, pp. 1–12, Jun. 2018, doi: https://doi.org/10.1016/j.apenergy.2018.03.010.

[3] E. A. Soto, L. B. Bosman, E. Wollega, and W. D. Leon-Salas, “Peer-to-peer energy trading: A review of the literature,” Applied Energy, vol. 283, p. 116268, Feb. 2021, doi: https://doi.org/10.1016/j.apenergy.2020.116268.

[4] H. Muhsen, A. Allahham, A. Al-Halhouli, M. Al-Mahmodi, A. Alkhraibat, and M. Hamdan, “Business Model of Peer-to-Peer Energy Trading: A Review of Literature,” Sustainability, vol. 14, no. 3, p. 1616, Jan. 2022, doi: https://doi.org/10.3390/su14031616.

[5] A. H. Lone and R. Naaz, "Demystifying Cryptography behind Blockchains and a Vision for Post-Quantum Blockchains," 2020 IEEE International Conference for Innovation in Technology (INOCON), Bangluru, India, 2020, pp. 1-6, doi: 10.1109/INOCON50539.2020.9298215.

[6] A. Arooj, M. S. Farooq, and T. Umer, “Unfolding the blockchain era: Timeline, evolution, types and real-world applications,” Journal of Network and Computer Applications, vol. 207, p. 103511, Nov. 2022, doi: https://doi.org/10.1016/j.jnca.2022.103511.

[7] A. Nadeem, “A survey on peer-to-peer energy trading for local communities: Challenges, applications, and enabling technologies,” Frontiers in Computer Science, vol. 4, Sep. 2022, doi: https://doi.org/10.3389/fcomp.2022.1008504.

[8] M. Mehdinejad, H. Shayanfar, and B. Mohammadi-Ivatloo, “Decentralized blockchain-based peer-to-peer energy-backed token trading for active prosumers,” Energy, p. 122713, Nov. 2021, doi: https://doi.org/10.1016/j.energy.2021.122713.

[9] D. Kirli et al., “Smart contracts in energy systems: A systematic review of fundamental approaches and implementations,” Renewable and Sustainable Energy Reviews, vol. 158, p. 112013, Apr. 2022, doi: https://doi.org/10.1016/j.rser.2021.112013.

[10] L. Toderean et al., "A Lockable ERC20 Token for Peer to Peer Energy Trading," 2021 IEEE 17th International Conference on Intelligent

Computer Communication and Processing (ICCP), Cluj-Napoca, Romania, 2021, pp. 145-151, doi: 10.1109/ICCP53602.2021.9733665.

[11] S. Seven et al., “Energy Trading on a Peer-to-Peer Basis between Virtual Power Plants Using Decentralized Finance Instruments,” Sustainability, vol. 14, no. 20, p. 13286, Oct. 2022, doi: https://doi.org/10.3390/su142013286.

[12] S. Ratka, F. Boshell, and A. Anisie, “Blockchain Innovation Landscape Brief,” International Renewable Energy Agency, Abu Dhabi, 2019. Available: https://www.irena.org/-/media/Files/IRENA/Agency/Publication/2019/Feb/IRENA_Landscape_Blockchain_2019.pdf?la=en&hash=1BBD2B93837B2B7BF0BAF7A14213B110D457B392

[13] F. Blum, B. Severin, M. Hettmer, P. Hückinghaus and V. Gruhn, "Building Hybrid DApps using Blockchain Tactics -The Meta-Transaction Example," 2020 IEEE International Conference on Blockchain and Cryptocurrency (ICBC), Toronto, ON, Canada, 2020, pp. 1-5, doi: 10.1109/ICBC48266.2020.9169423.

Appendix

A.1 Code Repository

Github: https://github.com/BarrySauce/FYP-A1068

A.2 Experiment Data of Relayer on Public Blockchain

Meta-Transaction that batched 3 payment requests was tested first. Experiment was carried out 3 times for each public testnet, 3 Meta-Transactions were acquired to measure the average efficiency and cost. The link to the transaction detail on the blockchain explorer, time consumed, cost incurred are listed in the table below. The cost was measured according to the USD price of the native token of the respective testnet.

Ethereum Goerli testnet

Native Token: Ether (ETH)

Price: 1907 USD per ETH

Experiment No.	Meta-Transaction Link	Time (second)	Cost (USD)
1	Link 1	90s	7.12 USD
2	Link 2	94s	4.64 USD
3	Link 3	97s	5.09 USD
Average	N/A	93.67s	5.62 USD

Polygon Mumbai testnet

Native Token: MATIC

Price: 1.16 USD per MATIC

Experiment No.	Meta-Transaction Link	Time (second)	Cost (USD)
1	Link 1	76s	0.000723 USD
2	Link 2	78s	0.000374 USD
3	Link 3	77s	0.000380 USD
Average	N/A	77s	0.000489 USD

Arbitrum Goerli testnet

Native Token: Arbitrum Goerli (AGOR), equivalent to Ether (ETH)

Price: 1907 per AGOR

Experiment No.	Meta-Transaction Link	Time (second)	Cost (USD)
1	Link 1	68s	0.42 USD
2	Link 2	65s	0.39 USD
3	Link 3	66s	0.38 USD
Average	N/A	66.33s	0.397 USD

Optimism Goerli testnet

Native Token: Optimism Ether (ETH), equivalent to Ether (ETH)

Price: 1907 per ETH

Experiment No.	Meta-Transaction Link	Time (second)	Cost (USD)
1	Link 1	64s	3.16 USD
2	Link 2	65s	2.46 USD
3	Link 3	67s	2.22 USD
Average	N/A	65.33s	2.61 USD

Then Meta-Transaction that batched 6 payment requests was tested. Experiment was carried out 2 times for each public testnet, 2 Meta-Transactions were acquired to measure the average efficiency and cost. The link to the transaction detail on the blockchain explorer, time consumed, cost incurred are listed in the table below. The cost was measured according to the USD price of the native token of the respective testnet.

Ethereum Goerli testnet

Native Token: Ether (ETH)

Price: 1907 USD per ETH

Experiment No.	Meta-Transaction Link	Time (second)	Cost (USD)
1	Link 1	93s	19.16 USD
2	Link 2	92s	21.90 USD
Average	N/A	92.5s	20.54 USD

Polygon Mumbai testnet

Native Token: MATIC

Price: 1.16 USD per MATIC

Experiment No.	Meta-Transaction Link	Time (second)	Cost (USD)
1	Link 1	77s	0.0008 USD
2	Link 2	81s	0.0007 USD
Average	N/A	79s	0.00075 USD

Arbitrum Goerli testnet

Native Token: Arbitrum Goerli (AGOR), equivalent to Ether (ETH)

Price: 1907 per AGOR

Experiment No.	Meta-Transaction Link	Time (second)	Cost (USD)
1	Link 1	65s	0.58 USD
2	Link 2	63s	0.65 USD
Average	N/A	64s	0.62 USD

Optimism Goerli testnet

Native Token: Optimism Ether (ETH), equivalent to Ether (ETH)

Price: 1907 per ETH

Experiment No.	Meta-Transaction Link	Time (second)	Cost (USD)
1	Link 1	65s	4.09 USD
2	Link 2	63s	3.23 USD
Average	N/A	64s	3.66 USD